

DevSecOps Interview Q&A — 120 Questions

Study set for the Analog Devices Staff DevSecOps pipeline (Casey Galvin screen + whatever technical round follows). Written in English because the real interview will be. Ordered by how likely each topic is to come up, weighted toward the tools practiced hands-on in this crash course ([devsecops-crash-course.md](#), [devsecops-scan-report.md](#)) and the exact acronyms named in the job posting (SAST/SCA/SBOM/DAST/SSDLC). AWS and compliance sections are conceptual/definitional where there's a real gap — never claim hands-on experience that isn't there; the honest "I understand X, haven't operated it" answer is in the mix on purpose.

1. SAST — Static Application Security Testing (15)

1 What is SAST?

Static Application Security Testing analyzes source code without executing it, looking for known vulnerability patterns by parsing the code into an AST and matching structural patterns against it.

2 How does SAST differ from a plain text-based linter?

SAST tools like Semgrep parse code into an abstract syntax tree and match structural patterns, not literal strings — they understand code semantics (variable use, control flow), not just text.

3 What are SAST's main limitations?

Coverage depends entirely on the loaded ruleset. I proved this myself: Semgrep's `auto` config caught `subprocess.call(..., shell=True)` as command injection but missed the semantically identical `os.system("cmd " + var)` because no rule matched that exact syntactic pattern.

4 How do you close a SAST coverage gap?

Write a custom rule. I wrote a Semgrep YAML rule matching `os.system("..." + $VAR)` — running both rulesets together went from 3 to 4 findings on the same file. Takes minutes; you don't wait for the vendor's rule registry to catch up.

5 Does a clean SAST scan mean the code is secure?

No — it means no loaded rule matched. Zero findings proves absence of *known patterns*, not absence of vulnerabilities.

6 What's a common SAST false-negative pattern?

Semantically equivalent code written differently — the same command-injection bug via two different APIs is a real example I hit personally.

7 What's a common SAST false-positive source, and how is it handled?

Overly broad patterns matching safe code. Teams tune rule specificity or use inline suppressions/allowlists — but suppressions need review, since they can hide real issues introduced later.

8 Is SAST a substitute for code review?

No. It's one layer — it consistently misses semantically-equivalent-but- syntactically-different bugs that a human reviewer might still catch.

9 Where does SAST fit in the pipeline?

As early as possible — ideally on every commit/PR, before merge, while the author still has full context.

10 Should SAST findings block a PR merge?

Depends on maturity. For code under your control, yes for real findings — they're fixable immediately. For legacy code with existing debt, teams often start report-only until the backlog is triaged, or nobody can merge anything.

11 What did you personally build and run with SAST?

Installed and ran Semgrep against a real repo (zero findings, clean code) and against a deliberately vulnerable demo file with command injection, a weak hash, and a hardcoded secret. Diagnosed the coverage gap, wrote and validated a custom rule, confirmed the before/after finding count.

12 What format do Semgrep rules use?

YAML, with a `pattern` field written in a DSL that resembles the target language itself, matched structurally against the code's AST.

13 Can SAST find hardcoded secrets?

Some SAST tools include secret-detection rules — Semgrep flagged a fake Stripe key via a built-in rule — but dedicated secret scanners are usually more complete for that job, especially across git history.

14 What's the risk of relying only on a vendor's default ruleset?

Coverage blind spots specific to your codebase's idioms never get caught. You have to actively audit for what a generic ruleset misses and supplement it.

15 How would you introduce SAST into a codebase with zero prior security tooling?

Start non-blocking/report-only, triage the existing backlog by severity, fix or explicitly accept each finding, then flip it to a blocking gate — otherwise the first blocking run makes every PR unmergeable.

2. DAST — Dynamic Application Security Testing (12)

16 What is DAST?

Dynamic Application Security Testing attacks a running application from the outside, over HTTP, simulating a real attacker. It never reads source code.

17 How does a DAST tool work mechanically?

It runs a proxy that first spiders/crawls the app to discover routes, then replays captured requests with variations (missing headers, simple payloads) and audits the responses.

18 What can DAST catch that SAST cannot?

Runtime/config issues that don't exist as a line of source code. I ran OWASP ZAP against a 12-line FastAPI app with zero logic bugs and it still found 3 missing security headers — a purely runtime characteristic.

19 What can DAST NOT catch that SAST can?

Code paths DAST never exercises because it doesn't know which inputs trigger them — e.g. a command-injection bug behind a specific parameter value it never happened to send.

20 What's a "baseline scan" in OWASP ZAP?

A fast, largely passive scan — mostly observes traffic and runs known-check rules rather than actively attacking. Good for CI: quick and non-destructive.

21 Should DAST block a CI pipeline?

Often not, at least initially — a baseline finding (a missing header) may need an architecture decision, not a one-line fix. Turning it into a hard gate without triage blocks every merge over things not yet assessed.

22 What did you personally fix using DAST?

Ran ZAP baseline against a running container, got 3 WARN. Added a FastAPI middleware setting `X-Content-Type-Options`, `Cross-Origin-Resource-Policy`, and `Cache-Control` — rebuilt, reran, down to 1 WARN. That last WARN is the expected side effect of `Cache-Control: no-store` — a conscious tradeoff, not a bug.

23 What's an authenticated DAST scan, and why does it matter?

A scan that logs in first so it can test routes behind auth — most real functionality lives there. I didn't set this up (the demo app has no auth) — a real, acknowledged gap.

24 DAST vs a manual pentest — what's the difference?

DAST is automated and repeatable but shallow/generic. A pentest is manual, creative, and can chain findings into a real exploit. DAST is baseline hygiene, not a pentest substitute.

25 Why run DAST against staging rather than production?

It sends real traffic/attacks — running against production risks disruption, data creation, or tripping real alerts/rate limits.

26 What's a typical DAST finding category?

Missing security headers, verbose error messages leaking stack traces, weak session/cookie flags, outdated JS libraries, basic injection probes.

27 How do you prioritize DAST findings?

By exploitability and exposure — an endpoint leaking version info is lower risk than one accepting unsanitized input that reaches a database or shell.

3. SCA — Software Composition Analysis (12)

28 What is SCA?

Software Composition Analysis audits third-party dependencies: it extracts name+exact version from a manifest/lockfile and cross-references known CVE databases. It never analyzes the dependency's actual code.

29 SCA vs SAST — core difference?

SAST audits code you wrote; SCA audits code you imported.

30 What did you find running SCA for real?

Against a `requirements.txt` pinned to old versions (`requests==2.6.0`, `pyyaml==5.1`, `urllib3==1.24.1`), Trivy found 20 real CVEs — 3 CRITICAL, including a PyYAML RCE via `FullLoader` (CVE-2019-20477).

31 How did you remediate, and what surprised you?

Pinned each library to its fixed version and rescanned — 20 CVEs down to 0. The surprise: my first fix used a version *range* (`>=`), and Trivy didn't scan at all (`Number of language-specific files: 0`).

32 Why does a version range break SCA scanning?

The scanner needs to know the precise installed version to look it up in the vulnerability DB. A range doesn't resolve to one version until installed, so most SCA tools skip unpinned manifests entirely.

33 What's the practical lesson for real CI pipelines?

Dependencies must be pinned/locked for SCA to mean anything — loose ranges make the security gate blind, not just imprecise.

34 Should SCA findings block a merge?

Generally yes for CRITICAL/HIGH in a project under your control — a version bump is low-cost and immediate.

35 What tool automates dependency version bumps?

Dependabot or Renovate. I configured `dependabot.yml` for pip, the Dockerfile base image, and GitHub Actions versions, running weekly.

36 What's a transitive dependency, and why does it complicate SCA?

A dependency of your dependency, not something you declared directly. A vulnerable transitive package is harder to spot, and sometimes can't be bumped without also bumping (or forking) the direct dependency pinning it.

37 Give an example of a fix that didn't work cleanly.

Bumping `fastapi` to pull a patched `starlette` worked (removed 3 HIGH CVEs). But force-upgrading `pip` / `setuptools` / `wheel` to fix 2 remaining CVEs just swapped them for two *different* CVEs, net zero gain. I reverted it rather than report a false win.

38 How do you verify a fix actually worked?

Always rescan and compare the actual finding list/count — never assume an upgrade reduces risk without re-running the scanner.

39 SCA vs SBOM — how do they relate?

SCA does the vulnerability lookup; SBOM is the underlying inventory data that makes lookups fast without a full rescan. Some SCA pipelines consume an existing SBOM instead of re-parsing manifests from scratch.

4. SBOM — Software Bill of Materials (10)

40 What is an SBOM?

A formal, machine-readable inventory of every component in a software artifact (libraries, versions, sometimes licenses and build-time actions), typically in CycloneDX or SPDX format.

41 Why does SBOM matter when a new CVE drops?

Instead of rescanning your entire fleet, you query already-generated SBOMs to know in minutes which artifacts contain the affected component and version.

42 What did you do to actually test that value?

Simulated a hypothetical new CVE in `anyio < 4.16.0`, then queried an already-generated `sbom.json` instead of rerunning Trivy — found `anyio 4.15.1` (exposed) in milliseconds.

43 What tool and format did you use?

Syft, output as CycloneDX JSON.

44 Does scan scope change what's captured?

Significantly — a directory (source-only) scan of a real repo gave 40 components (including GitHub Actions used in CI); a full container image scan gave 2,872 components, because it captures every file in the image, not just language packages.

45 Is SBOM a security gate?

No — it's inventory, not a pass/fail check. It's published as a CI artifact for later reference, never blocking anything.

46 Why is SBOM a regulatory topic in some industries?

A 2021 US Executive Order pushed federal software supply-chain requirements, and the FDA has required SBOMs in medical device software submissions since 2023 — for a SaMD product line, it can be a submission requirement, not just best practice.

47 CycloneDX vs SPDX?

Both are standard SBOM formats. CycloneDX originated in the OWASP ecosystem with a security/vulnerability focus; SPDX originated in the Linux Foundation with a stronger license-compliance focus. Many tools emit either.

48 What non-obvious thing showed up in your SBOM?

The repo-level SBOM included GitHub Actions used in CI (e.g. `actions/checkout`) as components — SBOM covers supply-chain elements beyond runtime language dependencies.

49 How would you operationalize SBOM at scale?

Generate one automatically on every build/release as a pipeline artifact, store them centrally indexed by version, and build simple tooling to query across all of them by component name — that's what turns "SBOM exists" into "SBOM is useful."

5. Container Security / Scanning (10)

50 Why scan a container image instead of just the app's code?

The final image includes the OS base layer and its packages — code review never sees that. Measured for real: the full `python:3.11` base image had 654 CRITICAL/HIGH vulnerabilities, almost entirely from the Debian OS, not the app.

51 What did switching base images achieve?

Moving to `python:3.11-slim` dropped it to 59 (a ~91% reduction) without touching a line of app code.

52 How does Trivy scan a container image mechanically?

It unpacks the image layer by layer, lists installed OS packages (dpkg/apk/rpm metadata) plus language-level packages inside, and cross-references the same CVE database used for filesystem scans.

53 What further step did you take beyond `-slim`, and what was the result?

I first tried a multi-stage build into a `distroless` runtime — it dropped to 49, but still with 2 CRITICAL. Then my CI gate blocked the plain slim image, so I measured the simplest option I had skipped: `apt-get upgrade` in the Dockerfile (the CRITICAL `perl-base` findings had fixes available all along) plus removing `setuptools` / `wheel`, which the app doesn't use at runtime. That gave 44 findings, 0 CRITICAL, and 0 with `--ignore-unfixed` — better than distroless on every number, and it keeps a shell. Lesson: measure the simple option before the sophisticated one.

54 What's the tradeoff of distroless, and did you end up using it?

No shell inside the running container — `docker exec sh` doesn't work, so debugging a live incident depends entirely on logs and external observability. I didn't adopt it: the hardened slim image beat it on the numbers, so paying the debuggability cost bought nothing. Distroless makes sense when the residual attack surface actually matters more than operability — I measured instead of assuming.

55 How do you validate a hardened image still works?

Actually run it and test real behavior — I curled the `/health` endpoint and confirmed a 200 with the expected security headers before trusting the vulnerability-count improvement.

56 What do you do with residual vulnerabilities that have no available fix?

Confirm it in the scanner output itself (Trivy reports `Fixed Version: None`) — that's an upstream gap, not an effort gap. It becomes a monitoring item.

57 How do you catch a new vulnerability in an image that never gets rebuilt?

A scheduled rebuild — I added a weekly cron trigger to CI specifically so the same base-image tag gets rebuilt and rescanned even without a code push, since the tag's *content* can be patched upstream without the tag name changing.

58 What's the blocking policy for container-scan findings?

CRITICAL/HIGH block the merge — same logic as SCA: usually fixable by a base-image or dependency change.

59 Give an example of an attempted fix that backfired.

Force-upgrading `pip` / `setuptools` / `wheel` to close 2 findings swapped them for two different ones from newly-vendored tooling — net zero. Reverted and documented instead of reported as fixed.

6. Secrets Management & Scanning (8)

60 What is secrets scanning?

Detecting hardcoded credentials — API keys, passwords, tokens — committed into source code or git history.

61 How does Gitleaks work mechanically?

It walks `git log` commit by commit, not just the current working tree, applying regex and entropy checks against known credential patterns plus custom rules.

62 Why doesn't deleting a leaked key from the file fix the problem?

The key still exists in the older commit's snapshot. I proved this in a throwaway repo: committed a fake key, "fixed" it in the next commit, and Gitleaks still flagged it, pointing at the earlier commit — `git show` on that commit returned the full key.

63 What's the correct remediation for a real leaked credential?

Rotate it immediately — treat the old value as permanently compromised regardless of what happens to the repo. Rewriting history is cleanup, not the fix, and it breaks existing clones.

64 Where should secrets scanning run?

Two points ideally: a local pre-commit hook (catches it before a commit exists) and CI (a safety net if the hook wasn't installed or was bypassed).

65 What did you build to enforce this locally?

A `.pre-commit-config.yaml` running the official Gitleaks hook on every commit. Tested both directions — blocked a real random AWS-format key (exit code 1, commit rejected) and let through `AKIAIOSFODNN7EXAMPLE`.

66 Why did that specific key pass, and what's the lesson?

`AKIAIOSFODNN7EXAMPLE` is AWS's own documentation placeholder, on Gitleaks' default allowlist so tutorials don't create noise. Lesson: every scanner ships default allowlists — never validate a hook using a vendor's canonical "example" credential, or you'll wrongly conclude it doesn't work.

67 Should secrets ever be committed "for a demo"?

No — even documentation describing a fake secret can trip real scanners (GitHub's own push protection blocked a commit of mine over a fake Stripe key pasted into a markdown doc). Redact or describe it; never paste anything matching a real credential's shape.

7. SSDLC & CI/CD Security Gates (10)

68 What is SSDLC?

Secure Software Development Lifecycle — not a tool, a *process*: security built into every phase (design, code, build, deploy, operate), not a single scan bolted on before release.

69 How do the individual tools map onto SSDLC phases?

SAST and secrets scanning run at code/commit time; SCA and container scanning run at build time; SBOM is generated at build/release time; DAST runs against a deployed environment.

70 Blocking vs non-blocking gate — what's the difference?

Blocking gates fail the pipeline and stop the merge/deploy; non-blocking gates report findings without stopping anything.

71 How do you decide which gates block?

By how directly fixable the finding is and how much control you have. In my pipeline: SAST, secrets, and SCA (CRITICAL/HIGH) block — fixable in code/deps you control. Container scan blocks on CRITICAL/HIGH. SBOM never blocks. DAST only reports, since a baseline finding can need an architecture decision.

72 What happens if every gate is blocking on day one in a legacy codebase?

Nobody can merge anything — the existing backlog has to be triaged (fixed, or explicitly accepted/suppressed) before the check becomes a hard gate.

73 How do you keep dependencies and base images from silently rotting?

Automate it — Dependabot for version bumps, plus a scheduled CI rebuild to catch security patches landing on the same tag without a version bump.

74 What's "shift-left" security?

Moving checks as early as possible in the pipeline — e.g. SAST/secrets scanning pre-commit rather than only at deploy time, so the person with full context fixes it immediately.

75 What proves a pipeline actually works end-to-end?

Running it against a real PR and watching the jobs execute. I pushed a trivial change and opened a PR, and the first real run found three problems the paper version hid: (1) I'd guessed a nonexistent action tag (`trivy-action@0.28.0` — the repo uses `v0.36.0`); (2) the container gate blocked the very findings I'd documented as accepted, forcing a real policy decision (block only what has a fix available, via `ignore-unfixed`); (3) the ZAP action failed with "Resource not accessible by integration" because the default `GITHUB_TOKEN` can't create issues — the scan itself had matched my local result exactly. None of that shows up until the pipeline actually runs.

76 How do you reduce alert fatigue across multiple security tools?

Tune each ruleset to actual risk tolerance, route only actionable findings to block merges, keep exploratory findings (like DAST baseline warnings) as reports until triaged.

77 What's a realistic first-90-days plan to add SSDLC gates to an ungated pipeline?

Land each scanner report-only first, triage the initial backlog by severity, fix or accept each item, then flip the highest-confidence/lowest-noise checks (secrets, SCA CRITICAL) to blocking first, and expand from there.

8. AWS Security (18)

78 What is IAM?

AWS's Identity and Access Management service — controls who (users, roles, services) can do what to which resources, via policies.

79 IAM user vs IAM role?

A user is a long-lived identity with its own credentials; a role is assumed temporarily with short-lived credentials. Roles are generally preferred for automation — no long-lived secret to leak.

80 What's least privilege in an IAM context?

Granting only the specific permissions a user/role/service needs — nothing broader "just in case."

81 What is KMS?

AWS Key Management Service — a managed service for creating and controlling encryption keys used to protect data at rest (and sometimes in transit).

82 How does KMS differ from encrypting something yourself?

KMS centralizes key lifecycle (rotation, access policy, audit logging via CloudTrail) instead of each app/service managing its own raw key material.

83 What is Cognito?

AWS's managed user identity/authentication service — sign-up, sign-in, and access control for apps, including federation with external identity providers.

84 ECS vs EKS?

Both run containers on AWS. ECS is AWS's own proprietary orchestrator; EKS is AWS's managed Kubernetes offering — same problem category, different control plane.

85 If you've only run self-managed Kubernetes, how do you speak to EKS?

Honestly: no hands-on with AWS's managed control plane, but real experience administering a Kubernetes control plane end-to-end (upgrades, node failure, storage, secrets, networking) — arguably more operational surface than only consuming a managed service.

86 What is AWS Lambda?

A serverless compute service — deploy a function, AWS runs it on demand without you managing the underlying server.

87 What security concerns are specific to Lambda?

The execution role's permissions (over-privileged functions are common), dependency vulnerabilities bundled into the deployment artifact, and environment variable handling for secrets.

88 What is RDS?

AWS's managed relational database service — AWS handles patching, backups, and failover infrastructure.

89 Security-relevant difference between RDS and a self-hosted database?

RDS handles OS/engine patching and backup infrastructure, but you're still responsible for network exposure (security groups/VPC), access control, and encryption configuration.

90 What is S3, and what's a classic S3 security mistake?

AWS's object storage service. Classic mistake: an accidentally public bucket (misconfigured policy/ACL) exposing sensitive data — a recurring root cause in real breach postmortems.

91 What is SQS?

AWS's managed message queue service, used to decouple services — a producer writes a message, a consumer processes it later, absorbing load spikes.

92 What is CloudFormation?

AWS's native infrastructure-as-code service — declare resources in a template, AWS provisions/updates them to match.

93 CloudFormation vs Terraform?

Same category (declarative IaC). CloudFormation is AWS-native, only manages AWS; Terraform is multi-cloud/provider-agnostic via its provider ecosystem.

94 What AWS service continuously checks for account misconfigurations?

AWS Config (compliance rules) or Security Hub (aggregated findings) for misconfig; GuardDuty for threat detection specifically.

95 If production runs on Lightsail today, how do you talk about evolving to a standard AWS security posture?

Frame it as future evolution, not existing experience: "today it runs on Lightsail; at greater scale I'd evaluate VPC-isolated EC2/ECS with proper IAM roles, KMS-backed encryption, and CloudWatch monitoring" — shows the destination without inventing claims you don't have.

9. Compliance & Regulated Software (15)

96 What is HIPAA, and why does it matter to a DevSecOps engineer?

The US law governing protection of health data (PHI). It constrains how systems log, store, transmit, and grant access to that data — engineering decisions have direct compliance consequences.

97 What is SaMD?

Software as a Medical Device — an FDA regulatory category for software that itself functions as a medical device, with its own lifecycle and risk-management requirements.

98 How does SaMD differ from software embedded in a regulated physical device?

SaMD IS the device from a regulatory standpoint; embedded software is regulated as part of the hardware's overall approval — different regulatory pathway.

99 What is IEC 62304?

An international standard for medical device software lifecycle processes — defines required activities/documentation across the development lifecycle.

100 What is ISO 14971?

An international standard for risk management applied to medical devices — a structured process for identifying, evaluating, and controlling risk across the product's life.

101 How do IEC 62304 and ISO 14971 relate?

IEC 62304 governs the software lifecycle process; ISO 14971 governs the risk-management process feeding into decisions made throughout that lifecycle — complementary, not overlapping.

102 What is ISO 27001?

An international standard/certification for an organization's Information Security Management System — governance/process-level, not a per-product audit.

103 What is SOC 2?

A US audit framework/report attesting a service organization's controls meet defined trust criteria — commonly requested by enterprise customers during vendor due diligence.

104 ISO 27001 vs SOC 2 — practical difference?

ISO 27001 is a certification with an ongoing management system; SOC 2 is an audit report (Type I or Type II) usually requested contractually by US enterprise customers. Different audience, similar underlying intent.

105 Why does SBOM matter specifically for a SaMD product?

The FDA has required SBOMs in medical device software submissions since 2023 — for that product line it's a submission requirement, not just best practice.

106 What experience is relevant to regulated health data, even without SaMD-specific work?

Two years supporting a lab operating under HIPAA and equipment under clinical regulation — real exposure to the operational discipline regulated environments require, even without the SaMD/IEC 62304 process itself.

107 How would you approach a codebase under IEC 62304/FDA scope if you've never worked in that process before?

Ask explicitly what documentation/traceability the process requires before changing anything — the paper trail (requirements → design → test → risk assessment) is often as load-bearing as the code, and skipping it can invalidate a submission.

108 What's a "risk-based" approach in ISO 14971 terms?

Prioritizing controls/mitigation effort by the severity and likelihood of potential harm, documented and revisited across the lifecycle — not a one-time checklist.

109 Why might a company be conservative about full CI/CD automation in a regulated product line?

Changes may need to trace to a formal risk assessment and validation record before release — full automation without a compliance gate can undermine the audit trail regulators expect.

110 What is OWASP, in one sentence?

A nonprofit/community publishing widely-referenced application security resources — most famously the OWASP Top 10.

10. OWASP Top 10 & AppSec Fundamentals (8)

111 What's in the OWASP Top 10, broadly?

Categories like broken access control, cryptographic failures, injection, insecure design, security misconfiguration, vulnerable/outdated components, authentication failures, integrity failures, insufficient logging, and SSRF — exact ranking shifts between revisions, categories recur.

112 What is command injection, concretely?

Untrusted input reaching a shell command without escaping/parameterization — e.g. `os.system("tar -czf backup.tar.gz " + filename)`, where an attacker-controlled `filename` injects shell syntax.

113 Why is `shell=True` risky in `subprocess` calls?

It spawns the command through a shell, so shell metacharacters in the input are interpreted, not passed as a literal argument. Fix: `shell=False` with arguments as a list.

114 Why is MD5 unsuitable for password hashing?

It's fast and unsalted by design, making brute-force/rainbow-table attacks cheap. Fix: a slow, salted algorithm like bcrypt, scrypt, or Argon2.

115 What's a security misconfiguration, concretely, from your own testing?

A missing `X-Content-Type-Options` header, or unnecessarily cacheable content on a sensitive endpoint — small runtime defaults that add up to real exposure.

116 Authentication vs authorization?

Authentication verifies who you are; authorization determines what you're allowed to do once identified. A broken-access-control bug is an authorization failure even with correct authentication.

117 What's a vulnerable/outdated components issue, in your own words?

Exactly what SCA catches — a dependency with a known CVE the team hasn't upgraded, sometimes for years (e.g. `pyyaml==5.1`, RCE-vulnerable since 2019).

118 Why does defense in depth matter across all these tools?

No single tool covers everything — SAST misses semantically-equivalent bugs, DAST misses unreachable code paths, SCA doesn't look at your own code, secrets scanning doesn't catch logic bugs. Overlapping layers catch what any one layer misses.

11. Wrap-up (2)

119 What's your biggest DevSecOps gap right now?

I hadn't worked hands-on with SAST/DAST/SCA/SBOM tooling formally before this — closed that by installing and running each tool against real code, deliberately provoking failures, and fixing what I could verify. Still open: rotating a credential after an actual production leak, and an authenticated DAST scan — both need conditions I didn't have available to practice safely.

120 How do you stay current on emerging security tooling?

Hands-on, not passive reading. When I hit an unfamiliar tool, I install it, run it against something real (or a deliberately vulnerable target), break it on purpose, and document what actually happened — not what the docs claim should happen.